보안 감사 잔여 8건 구현 계획서

> *작성일: 2026-03-26 기준 문서: `docs/DECISION_REQUIRED.md` 결정 사항 반영 현황: 25/33 (76%) → 목표: 33/33 (100%)*

결정 사항 요약

#	항목	결정
1-1	CUG PUBLIC 콘텐츠	**A. 허용** (홍보용 노출)
1-2	ACL 우선순위	**C. 계층적** — 단, 정책서 기준 "단체" 단일 개념으로 통합. Organization/Team ACL 분리 → 단체 ACL 통합 필요
1-3	CUG 캐시 무효화	**A. 즉시 무효화**
1-4	복합 정책	**B. 복합 정책 지원**
2-1	DAG 방어	**C. 앱 레벨 + Advisory Lock**
2-2	최대 깊이	**A. 5단계 유지**
3-1	Flyway 도입	**A. 즉시**
3-2	롤백 전략	**C. Fix-forward**
3-3	실행 방식	**A. 앱 시작 시 자동**
4-1	PII 대상	**A. 최소 (phone, birth, guardianPhone)**
4-2	암호화 방식	**C. 하이브리드** (email 결정적, 나머지 확률적)
4-3	키 관리	**A. 환경변수 → B. KMS 전환 예정**
4-4	키 로테이션	**C. 봉투 암호화** (KMS 전환 대비)
5-1	보호자 관계	**B. 1:N** (별도 테이블)
5-2	동의 프로세스	**C. 본인 인증 연동** (PASS 등)
5-3	제한 범위	**A. 결제 한도만**
5-4	미성년자 기준	**A. 14세 미만**
6-1	모더레이션 방식	**C. 하이브리드** (기본 사후, 경고 이력자 사전)
6-2	신고 카테고리	**B. 확장 9종**
6-3	모더레이터 권한	**A. 조직 내 모더레이터**
6-4	자동 모더레이션	**C. AI 기반** (외부 서비스)
7-1	메시지 브로커	**B. RabbitMQ**
7-2	Outbox 패턴	**B. Best-effort → 추후 C. Saga 전환**
7-3	이벤트 스키마	**B. JSON Schema**
8-1	탈퇴 처리	**A. 이벤트 기반** (DATA-002 선행)

8-2	테이블별 방침	문서 제안 그대로
8-3	유예 기간	**A. 즉시 삭제**

구현 순서 (의존성 기반)

```
Phase A (선행 없음, 병렬 가능)
├── BIZ-003/004: CUG 정책 정리 + ACL 통합
├── DATA-003: Advisory Lock 추가
├── DATA-005: Flyway 즉시 도입
└── SEC-008: PII 암호화 (환경변수 키)

Phase B (Phase A 완료 후)
├── DATA-002: RabbitMQ 이벤트 시스템
├── BIZ-005: GUARDIAN 보호자 (DB 테이블 + API)
└── BIZ-006: 커뮤니티 모더레이션

Phase C (Phase B의 DATA-002 완료 후)
└── DATA-001: 사용자 탈퇴 이벤트 처리
```

Phase A — 선행 의존성 없음

A-1. BIZ-003/004: CUG 정책 정리 + ACL "단체" 통합

배경

정책서(POCHAK_POLICY.md Section 4)에 "단일 Entity '단체'로 통합"이라고 명시되어 있으나, VideoAclService에서 Organization ACL과 Team ACL을 분리 처리 중. 이를 정책서에 맞게 통합.

수정 대상 파일

파일	변경
`content-service/.../acl/service/VideoAclService.java`	`evaluateMembersOnlyAccess()` — Organization/Team 분리 체크 → 통합 단체 체크. 복합 정책 지원(SUBSCRIBERS + MEMBERS_ONLY 조합). CUG PUBLIC 허용 정책 확인 코멘트 추가
`content-service/.../acl/entity/VideoAcl.java`	policy JSONB 스키마 문서화. `allowedOrganizations`/`allowedTeams` → `allowedGroups` 통합
`content-service/.../membership/entity/Membership.java`	TargetType 유지하되 ACL 체크 시 통합 조회
`content-service/.../membership/repository/MembershipRepository.java`	`findByUserIdAndTargetTypeInAndActiveTrue(userId, List.of(ORG, TEAM))` 추가
`content-service/.../organization/service/OrganizationService.java`	CUG 변경 메서드에 `@CacheEvict(value = "acl", allEntries = true)` 추가

ACL 계층적 체크 로직 (결정 1-2: C)

```
1. 단체(Organization) 레벨 ACL 체크
   └── 해당 단체 또는 하위 단체 소속 여부 (DAG 계층 활용)
```

```
2. 역할(Role) 레벨 ACL 체크
   └── 단체 내 역할(ADMIN, MANAGER, COACH, PLAYER 등) 확인
```

- Organization과 Team은 같은 organizations 테이블의 DAG 노드 → "단체" 소속 확인 = 해당 노드 또는 하위 노드의 membership 존재 확인

복합 정책 구조 (결정 1-4: B)

```json
{
  "defaultPolicy": "MEMBERS_ONLY",
  "additionalPolicies": ["SUBSCRIBERS"],
  "combineMode": "AND",
  "allowedGroups": [101, 201, 305],
  "allowedRoles": ["COACH", "PLAYER"]
}
```

- `combineMode: AND` → SUBSCRIBERS이면서 MEMBERS_ONLY 모두 만족
- `combineMode: OR` → 하나만 만족해도 허용

A-2. DATA-003: Advisory Lock 기반 DAG 동시성 보호

수정 대상 파일

파일	변경
`content-service/.../organization/service/OrganizationService.java`	`validateParentRelationship()` 호출 전 `SELECT FOR UPDATE` + pg_advisory_lock 추가
`content-service/.../organization/repository/OrganizationRepository.java`	`@Lock(PESSIMISTIC_WRITE) findById()` 오버로드 또는 native query 추가

구현 상세

```java
@Transactional
public void updateParent(Long orgId, Long newParentId) {
    // Advisory lock on organization hierarchy modification
    entityManager.createNativeQuery("SELECT pg_advisory_xact_lock(:lockKey)")
        .setParameter("lockKey", orgId.hashCode())
        .getSingleResult();

    Organization org = organizationRepository.findById(orgId)
        .orElseThrow(...);
    validateParentRelationship(org, newParentId); // 기존 앱 레벨 검증
    org.setParent(newParentRepository.findById(newParentId).orElse(null));
}
```

- 최대 깊이 5단계 유지
- pg_advisory_xact_lock → 트랜잭션 종료 시 자동 해제

A-3. DATA-005: Flyway 즉시 도입

수정 대상 파일

파일	변경
5개 서비스 `build.gradle.kts`	`implementation("org.flywaydb:flyway-core")` + `implementation("org.flywaydb:flyway-database-postgresql")` 추가

5개 서비스 `application.yml`	`spring.flyway` 설정 추가, `ddl-auto: update` → `validate` 전환
`db/migrations/`	서비스별 디렉토리 분리 또는 스키마별 prefix 관리

Flyway 설정 (각 서비스)

```yaml
spring:
  flyway:
    enabled: true
    baseline-on-migrate: true
    baseline-version: "0"
    locations: classpath:db/migration
    schemas: [identity]  # 서비스별 스키마
    default-schema: identity
  jpa:
    hibernate:
      ddl-auto: validate  # update → validate 전환
```

마이그레이션 파일 재배치

```
services/pochak-identity-service/src/main/resources/db/migration/
  ├── V001__create_identity_schema.sql (V001에서 identity 부분 추출)
  ├── V011__signup_account_system.sql
  ├── V012__fix_identity_schema.sql
  └── ...

services/pochak-content-service/src/main/resources/db/migration/
  ├── V002__create_content_schema.sql
  ├── V006__organization_simplification_and_abac.sql
  └── ...
```

- 각 서비스는 자기 스키마의 마이그레이션만 관리
- `baseline-on-migrate: true` → 기존 DB에 대해 baseline 자동 설정
- 롤백: Fix-forward 방식 (문제 발생 시 새 마이그레이션으로 수정)

A-4. SEC-008: PII 암호화

수정 대상 파일

파일	변경
`common-lib` 신규	`EncryptionUtil.java` — AES-256-GCM 유틸
`common-lib` 신규	`DeterministicEncryptConverter.java` — email용 결정적 암호화 JPA Converter
`common-lib` 신규	`ProbabilisticEncryptConverter.java` — phone/birth용 확률적 암호화 JPA Converter
`common-lib` 신규	`EncryptionKeyProvider.java` — 환경변수 기반 키 제공 (추후 KMS 전환)
`identity-service/.../user/entity/User.java`	PII 필드에 `@Convert(converter = ...)` 추가
신규 마이그레이션	기존 평문 데이터 암호화 배치

암호화 설계

키 관리 (봉투 암호화 준비)

```
KEK (Key Encryption Key): 환경변수 PII_MASTER_KEY (32바이트)
  └── DEK (Data Encryption Key): 랜덤 생성, KEK로 암호화하여 DB/파일에 저장
      └── 데이터 암호화: DEK로 AES-256-GCM
```

- 초기: KEK = 환경변수, DEK = 앱 시작 시 생성 후 파일 저장
- 전환: KEK = AWS KMS, DEK 재암호화만 하면 데이터 재암호화 불필요

필드별 적용

필드	방식	이유
email	결정적 (AES-256-SIV)	로그인/중복 체크 시 WHERE 검색 필요
phoneNumber	확률적 (AES-256-GCM)	검색 빈도 낮음, 높은 보안
birthDate	확률적 (AES-256-GCM)	검색 빈도 낮음
guardianPhone	확률적 (AES-256-GCM)	검색 빈도 낮음

DB 컬럼 변경

```sql
-- 암호화 데이터는 base64 인코딩되므로 길이 증가
ALTER TABLE identity.users ALTER COLUMN email TYPE VARCHAR(500);
ALTER TABLE identity.users ALTER COLUMN phone_number TYPE VARCHAR(500);
ALTER TABLE identity.users ALTER COLUMN guardian_phone TYPE VARCHAR(500);
-- birthDate는 VARCHAR로 변환 (암호화 후 날짜 타입 불가)
ALTER TABLE identity.users ALTER COLUMN birth_date TYPE VARCHAR(500) USING birth_date::TEXT;
```

Phase B — Phase A 완료 후

B-1. DATA-002: RabbitMQ 이벤트 시스템

인프라 변경

파일	변경
`infra/docker-compose.yml`	RabbitMQ 컨테이너 추가 (`rabbitmq:3-management-alpine`)
5개 서비스 `build.gradle.kts`	`spring-boot-starter-amqp` 추가
5개 서비스 `application.yml`	RabbitMQ 연결 설정
`common-lib`	`RabbitMqEventPublisher.java` — InMemoryEventPublisher 대체

RabbitMQ 설정

```yaml
# docker-compose.yml
pochak-rabbitmq:
  image: rabbitmq:3-management-alpine
  ports:
    - "5672:5672"    # AMQP
    - "15672:15672"  # Management UI
  environment:
    RABBITMQ_DEFAULT_USER: pochak
```

```
    RABBITMQ_DEFAULT_PASS: ${RABBITMQ_PASSWORD}
  volumes:
    - pochak-rabbitmq-data:/var/lib/rabbitmq
```

Exchange/Queue 설계

```
Exchange: pochak.events (topic exchange)
  ├── Queue: identity.events  ← Routing key: user.*
  ├── Queue: content.events   ← Routing key: content.*, membership.*
  ├── Queue: commerce.events  ← Routing key: purchase.*, subscription.*, refund.*
  ├── Queue: operation.events ← Routing key: reservation.*, streaming.*
  └── Queue: admin.events     ← Routing key: *.* (모든 이벤트 감사 로깅)
```

JSON Schema 이벤트 검증

```java
// common-lib에 추가
@Component
public class EventSchemaValidator {
    // src/main/resources/schemas/ 디렉토리에 JSON Schema 파일 배치
    // 발행 시 검증, 수신 시 검증
}
```

Best-effort 발행 방식

```java
@Component
public class RabbitMqEventPublisher implements EventPublisher {
    @Override
    @Transactional(propagation = Propagation.REQUIRES_NEW)
    public void publish(DomainEvent event) {
        try {
            rabbitTemplate.convertAndSend("pochak.events", event.getRoutingKey(), event);
        } catch (AmqpException e) {
            log.error("Failed to publish event: {}", event.getEventId(), e);
            // Best-effort: 실패 시 로그만 남기고 트랜잭션은 유지
            // 추후 Saga 전환 시 보상 트랜잭션 추가
        }
    }
}
```

B-2. BIZ-005: GUARDIAN 보호자

신규 파일

파일	내용
identity-service 신규 엔티티	GuardianRelationship.java — guardian_id, minor_id, status, consent_method, consented_at
identity-service 신규 리포지토리	GuardianRelationshipRepository.java
identity-service 신규 서비스	GuardianService.java — 연결/동의/해제 로직
identity-service 신규 컨트롤러	GuardianController.java — REST API

신규 마이그레이션	`guardian_relationships` 테이블 + 인덱스

데이터 모델

```
CREATE TABLE identity.guardian_relationships (
    id BIGSERIAL PRIMARY KEY,
    guardian_id BIGINT NOT NULL REFERENCES identity.users(id),
    minor_id BIGINT NOT NULL REFERENCES identity.users(id),
    status VARCHAR(20) NOT NULL DEFAULT 'PENDING',  -- PENDING, VERIFIED, REVOKED
    consent_method VARCHAR(20) NOT NULL,  -- PASS_AUTH, KAKAO_AUTH, etc.
    consented_at TIMESTAMPTZ,
    monthly_payment_limit INTEGER DEFAULT 0,  -- 결제 한도 (원)
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMPTZ,
    UNIQUE(guardian_id, minor_id)
);
```

API 엔드포인트

```
POST   /api/v1/guardians/request      — 보호자 연결 요청 (미성년자 → 보호자)
POST   /api/v1/guardians/verify       — 본인 인증 동의 (PASS 연동)
GET    /api/v1/guardians/minors       — 보호자의 미성년자 목록
GET    /api/v1/guardians/my-guardian   — 미성년자의 보호자 정보
PUT    /api/v1/guardians/{id}/limit   — 결제 한도 변경
DELETE /api/v1/guardians/{id}         — 관계 해제
```

미성년자 판별 (14세 미만)

```java
// User 엔티티 또는 GuardianService
public boolean isMinorByAge(User user) {
    if (user.getBirthDate() == null) return false;
    return Period.between(user.getBirthDate(), LocalDate.now()).getYears() < 14;
}
```

결제 한도 연동

- Commerce Service의 PurchaseService에서 구매 전 Identity Service API 호출
- `GET /api/v1/guardians/check-limit?userId={minorId}&amount={amount}`
- Circuit Breaker 적용 (DATA-010 패턴 재사용)

B-3. BIZ-006: 커뮤니티 모더레이션

신규 파일

파일	내용
신규 엔티티	`PostReport.java` — 신고 기록
신규 엔티티	`ModerationAction.java` — 조치 기록
신규 Enum	`ReportCategory.java` — 9종 신고 카테고리
신규 Enum	`ModerationStatus.java` — PENDING, REVIEWING, RESOLVED, DISMISSED
신규 서비스	`ModerationService.java`

신규 컨트롤러	`ModerationController.java`
CommunityPost 수정	`moderationStatus` 필드 추가, `warningCount` 추가

신고 카테고리 (9종)

```
public enum ReportCategory {
    SPAM,           // 스팸/광고
    ABUSE,          // 욕설/비방
    HARASSMENT,     // 괴롭힘
    HATE_SPEECH,    // 혐오 발언
    SEXUAL,         // 선정적 콘텐츠
    VIOLENCE,       // 폭력적 콘텐츠
    COPYRIGHT,      // 저작권 침해
    MISINFORMATION, // 허위 정보
    OTHER           // 기타
}
```

하이브리드 모더레이션 로직

```
@Transactional
public CommunityPost createPost(CreatePostRequest request, Long userId) {
    User author = userService.getUser(userId);

    // 경고 이력 확인 → 사전 승인 필요 여부
    boolean requiresPreModeration = author.getWarningCount() >= 3
        || author.getCreatedAt().isAfter(LocalDateTime.now().minusDays(7)); // 신규 회원 7일

    CommunityPost post = CommunityPost.builder()
        .moderationStatus(requiresPreModeration ? ModerationStatus.PENDING :
ModerationStatus.APPROVED)
        .build();

    // AI 자동 모더레이션 (비동기)
    if (!requiresPreModeration) {
        moderationService.autoModerateAsync(post);
    }

    return communityPostRepository.save(post);
}
```

AI 자동 모더레이션

```
@Async
public void autoModerateAsync(CommunityPost post) {
    // 외부 AI 서비스 호출 (Perspective API 또는 자체 모델)
    ModerationResult result = aiModerationClient.analyze(post.getTitle(), post.getBody());

    if (result.getToxicityScore() > 0.8) {
        post.setModerationStatus(ModerationStatus.PENDING);
        post.setAutoFlagReason(result.getCategories().toString());
        communityPostRepository.save(post);

        // 단체 모더레이터에게 알림 (RabbitMQ 이벤트)
```

```
        eventPublisher.publish(new PostFlaggedEvent(post.getId(), post.getOrganizationId()));
    }
}
```

모더레이터 권한 (조직 내)

- Membership.MembershipRole이 ADMIN 또는 MANAGER인 사용자만 해당 조직 게시물 관리 가능
- `@PreAuthorize` 또는 서비스 레벨에서 조직 membership + 역할 체크

Phase C — DATA-002 완료 후

C-1. DATA-001: 사용자 탈퇴 이벤트 처리

이벤트 정의

```
public class UserWithdrawnEvent extends DomainEvent {
    private Long userId;
    private String emailHash;  // 재가입 방지용
    private LocalDateTime withdrawnAt;

    @Override
    public String getRoutingKey() { return "user.withdrawn"; }
}
```

각 서비스 이벤트 리스너

서비스	처리
Identity	PII 필드 초기화 (email→hash, phone→null, birth→null). status=WITHDRAWN. auth_accounts 삭제
Commerce	purchases: user_id 유지 (세법 5년 보관), entitlements: 삭제, wallets: 잔액 0
Content	community_posts: author_user_id → -1. memberships/follows/likes/comments: 삭제. view_histories: 삭제
Operation	진행 중 예약 자동 취소. 완료 예약: user_id → -1
Admin	audit_logs: 변경 없음 (영구 보관)

처리 방식: 즉시 삭제 (유예 없음)

```
// Identity Service — 탈퇴 요청 처리
@Transactional
public void withdrawUser(Long userId) {
    User user = userRepository.findById(userId).orElseThrow(...);

    // 1. PII 즉시 초기화
    String emailHash = DigestUtils.sha256Hex(user.getEmail());
    user.clearPii(emailHash);
    user.withdraw();
    userRepository.save(user);

    // 2. 이벤트 발행 → 다른 서비스가 정리
    eventPublisher.publish(new UserWithdrawnEvent(userId, emailHash, LocalDateTime.now()));

    // 3. 토큰 무효화
    refreshTokenRepository.deleteByUserId(userId);
```

```
    // Redis 블랙리스트에 추가 (JWT 즉시 무효화)
    redisTemplate.opsForValue().set("token-blacklist:" + userId, "withdrawn", Duration.ofDays(30));
}
```

마이그레이션 목록

번호	내용	Phase
V022	PII 암호화 컬럼 타입 변경 + DEK 저장 테이블	A
V023	Flyway baseline 정리 (각 서비스별)	A
V024	guardian_relationships 테이블	B
V025	post_reports + moderation_actions 테이블, community_posts에 moderation_status 추가	B
V026	RabbitMQ dead letter 테이블 (optional)	B
V027	사용자 탈퇴 지원 — User.clearPii() 관련 컬럼 조정	C

검증 계획

Phase A 검증

- [ ] CUG PUBLIC 콘텐츠: 비회원으로 CUG 단체의 PUBLIC 콘텐츠 접근 → 허용 확인
- [ ] ACL 통합: allowedGroups로 통합된 정책 → Organization/Team 구분 없이 단체 소속 확인
- [ ] 캐시 무효화: CUG 변경 후 즉시 ACL 캐시 갱신 확인
- [ ] Advisory Lock: 동시 조직 계층 변경 시 순환 방지 확인
- [ ] Flyway: 각 서비스 부팅 시 마이그레이션 자동 실행 + validate 모드 에러 없음
- [ ] PII 암호화: 암호화 후 로그인 (email 결정적 암호화), 프로필 조회 (phone 복호화)

Phase B 검증

- [ ] RabbitMQ: PurchaseCompletedEvent 발행 → Content Service 수신 확인
- [ ] GUARDIAN: 보호자 연결 → 결제 한도 설정 → 미성년자 결제 시 한도 초과 차단
- [ ] 모더레이션: 신고 접수 → 모더레이터 조치 → AI 자동 플래깅

Phase C 검증

- [ ] 사용자 탈퇴: 탈퇴 요청 → 5개 스키마 데이터 정리 확인
- [ ] PII 초기화: 탈퇴 후 email/phone/birth 조회 불가 확인
- [ ] 감사 로그: 탈퇴 후에도 audit_logs 보관 확인

예상 작업량

Phase	항목 수	신규 파일	수정 파일	마이그레이션
A	4건	~10	~25	2
B	3건	~15	~20	3
C	1건	~3	~10	1
합계	**8건**	**~28**	**~55**	**6**